VIEWS

Implementing Views

- Introduction to Views
- Creating and Managing Views
- Optimizing Performance by Using Views

What Is a View?

- A view is like a virtual table or a stored query
- A view is referenced the same way as a table

Employee (table)

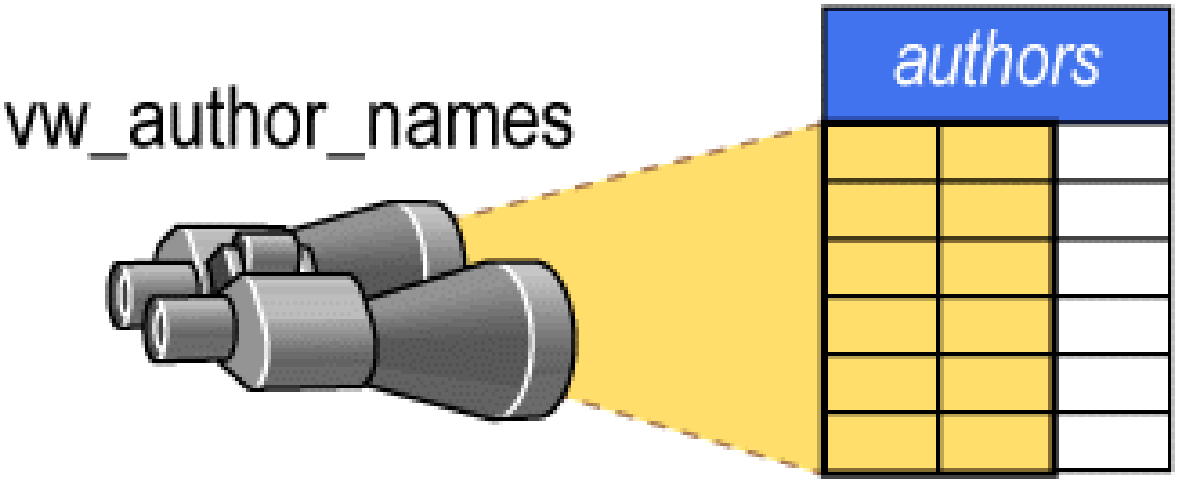
EmployeeID	LastName	FirstName	Title	…
287	Mensa-Annan	Tete	Mr.	…
288	Abbas	Syed	Mr.	…
289	Valdez	Rachel	NULL	…

vEmployee (view)

LastName	FirstName
Mensa-Annan	Tete
Abbas	Syed
Valdez	Rachel

View

- A view is a database object that consists of a stored **select** statement
- Used to:
  - Simplify construction of complex queries
  - Simplify user perceptions of the database
  - Limit access to data
  - Hide DB Objects



Advantages of Views

- Focus the data for a user
- Mask database complexity
- Simplify management of user permissions
- Improve performance
- Organize data for export to other applications

Types of Views

- **Standard views**

Combine data from one or more base tables into a new virtual table

- **Indexed views**

An indexed view has been computed and stored. You index a view by creating a unique clustered index on it

- **Partitioned views**

A partitioned view joins horizontally partitioned data from a set of tables across one or more servers

Syntax for Creating Views

- **Use the CREATE VIEW Transact-SQL statement:**

```
CREATE VIEW [ schema_name.] view_name [ (column [ ,...n ] ) ]
[WITH [ENCRYPTION] [SCHEMABINDING] [VIEW_METADATA] ]
AS select_statement [ ; ]
[ WITH CHECK OPTION ]
```

- **Restrictions on creating views:**

Cannot nest more than 32 levels deep

Cannot contain more than 1,024 columns

Cannot use COMPUTE, COMPUTE BY, or INTO

Cannot use ORDER BY without TOP

Considerations When Creating Views

Restriction	Description
Column Limit	• Total number of columns referenced in the view cannot exceed 1024
INTO	• Cannot be used with the SELECT statement in a view definition
Temporary table	• Cannot be referenced in a view
GO	• CREATE VIEW must be alone in a single batch
SELECT *	• Can be used in a view definition if the SCHEMABINDING clause is not specified

Syntax for Altering and Dropping Views

- **Alter by using the ALTER VIEW Transact-SQL statement:**

```
ALTER VIEW [ schema_name.]view_name [ (column [ ,...n ] ) ]
[WITH [ENCRYPTION] [SCHEMABINDING] [VIEW_METADATA] ]
AS select_statement [ ; ]
[ WITH CHECK OPTION ]
```

- **Drop by using the DROP VIEW Transact-SQL statement:**

```
DROP VIEW [ schema_name.]view_name [ ...,n ] [ ; ]
```

View Encryption

- **The WITH ENCRYPTION on CREATE VIEW T-SQL statement**

Encrypts view definition in **sys.syscomments** table

Protects view creation logic

```
CREATE VIEW [HumanResources].[vEmployee]
WITH ENCRYPTION AS SELECT
e.[EmployeeID],c.[Title],c.[FirstName],c.[MiddleName],
c.[LastName],c.[Suffix],e.[Title] AS [JobTitle],c.[Phone],c.[EmailAddress]
FROM [HumanResources].[Employee] e
  INNER JOIN [Person].[Contact] c
  ON c.[ContactID] = e.[ContactID]
```

Use WITH ENCRYPTION on ALTER VIEW to retain encryption

Using with check option

- **with check option** restricts how rows can be modified
  - Inserts attempting to add rows that the view could not see will fail
  - Updates attempting to modify rows so that the view could no longer see them will fail
- Example:

```
create view vw_kansas_authors
as
     select au_id, au_lname, au_fname, state, phone
            from authors
            where state = "KS"

            with check option
```

- Example of a modification that would fail:

```
update vw_kansas_authors
     set state = "MO" where state = "KS"
```

Views and Permissions

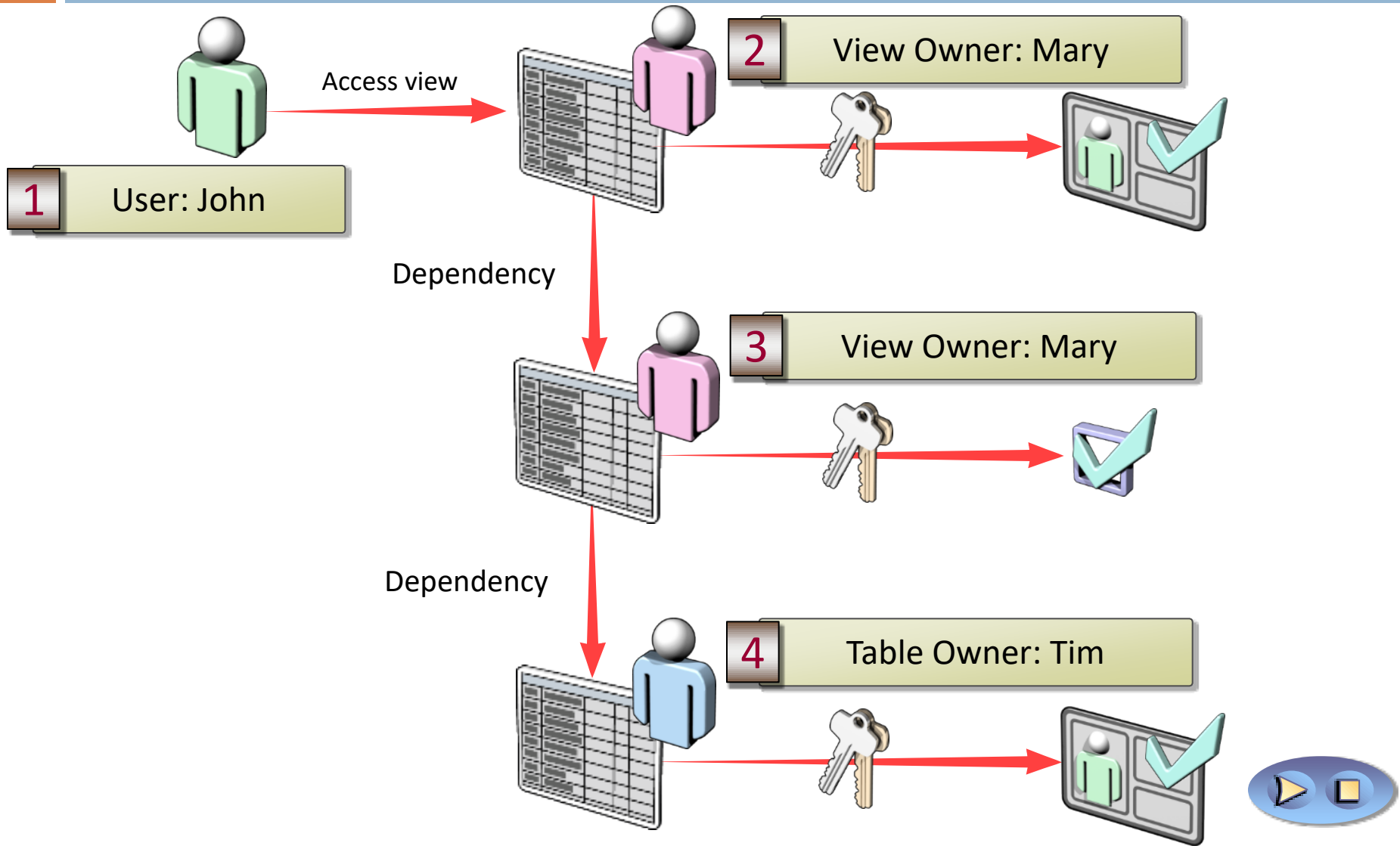
- To allow other people to use a view, you must grant them permission:

  grant [select | insert | update | delete | all]
  on *view_name*
  to *user_list*

- Example:

```
grant select
    on vw_california_authors
    to clerk1, clerk2, clerk3, clerk4
```

How Ownership Chains Affect Views



Modifying Data in a View

- **Views do not maintain a separate copy of data**
- **Updates to views modify the base tables**
- **Updates are restricted by using the WITH CHECK OPTION**

- **Restrictions:**

Cannot affect more than one base table

Cannot modify columns derived from aggregate functions

Cannot modify columns affected by GROUP BY, HAVING, or DISTINCT clauses

System Procedures for Views

- **sp_depends** {*table_name*, *view_name*}
  - When given a table, lists all objects (including views) in the same database that reference that table
  - When given a view, lists all tables in the same database referenced by the view
- **sp_help** *view_name*
  - Displays information about the specified view
- **sp_helptext** *view_name*
  - Displays the text used to create the specified view
- **sp_rename** *old_view_name*, *new_view_name*
  - Changes the name of a view

Optimizing Performance by Using Views

- Performance Considerations for Views
- Performance Considerations for Indexed Views
- What Is a Partitioned View?

Performance Considerations for Views

- **Views introduce performance overhead because views are resolved dynamically**
- **Indexed views and partitioned views can improve performance**
- **Nested views introduce risk of performance problems**

Review definition of unencrypted nested views

Use SQL Server Profiler to review performance

Indexed Views

An Indexed View is a view for which a unique clustered index has been created.

Indexed View Details

- Views can be indexed using CREATE INDEX
- Should not be used for views whose underlying data is updated frequently
- Columns must be listed explicitly
- Views must be created with the SCHEMABINDING option

Indexed View Example

Creating an Indexed View

```
create view v1
with schemabinding
as
select AddressID,AddressLine1 from Person.Address

create unique clustered index i
on v1(AddressID)

select * from v1
```

Performance Considerations for Indexed Views

- **A view with a unique clustered index**

Materializes view, improving performance

- **Use when:**

Performance gains outweigh maintenance overhead

Underlying data is modified infrequently

Queries perform a significant number of joins

Notes:

- Schema binding must be used on indexed view
- First index on view must be unique

Partitioned Views

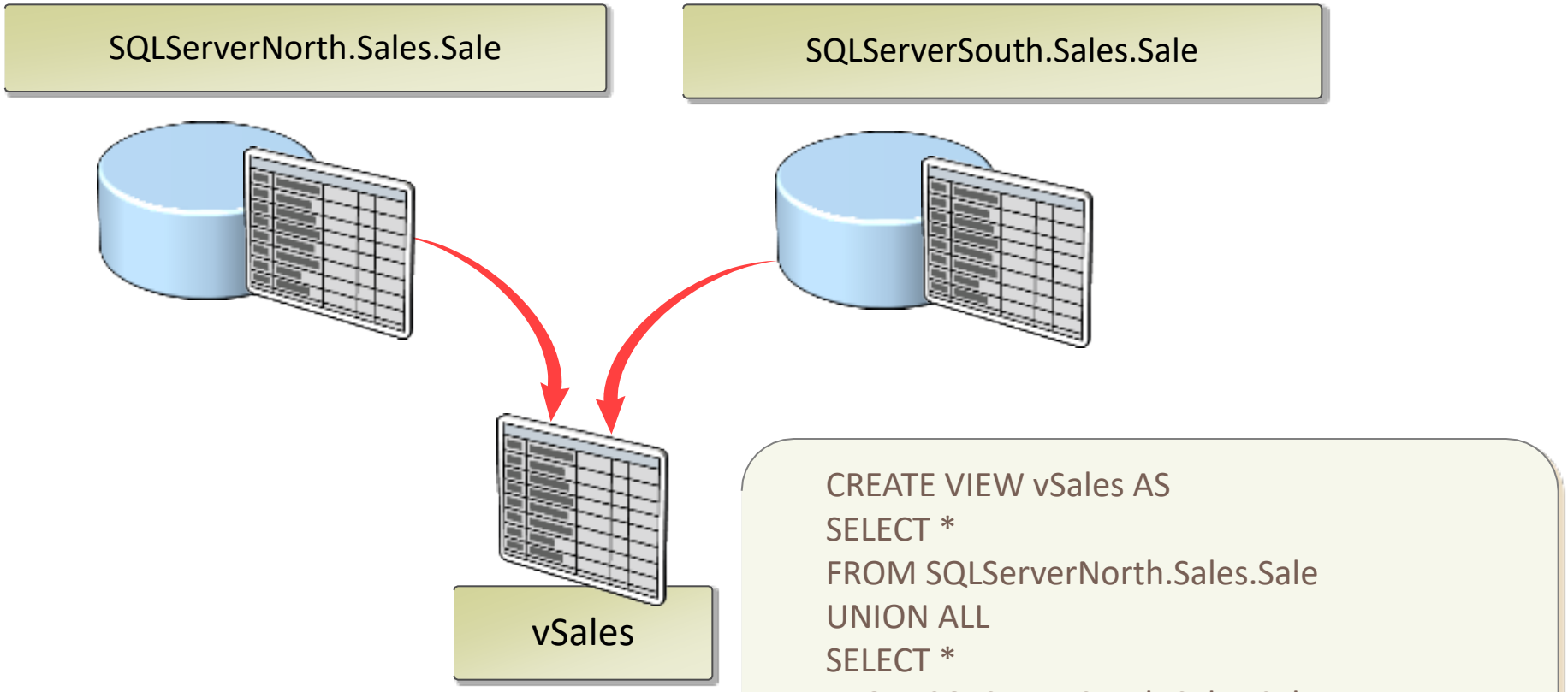
A partitioned view joins horizontally partitioned data from a set of member tables across one or more servers, making the data appear as if from one table.

Partitioned View Details

- Allows the data in a large table to be split into smaller member tables.
- Data can be partitioned between the member tables based on ranges of data values.
- Partitioned Views make it easier to maintain the member tables independently.

What Is a Partitioned View?

Joins partitioned data from set of tables across servers



Partitioned View Example

Creating a Partitioned View

```
CREATE TABLE May1998sales
(OrderID INT,
 CustomerID INT NOT NULL,
 OrderDate DATETIME NULL
CHECK (DATEPART(yy, OrderDate) = 1998),
 OrderMonth INT
CHECK (OrderMonth = 5),
 DeliveryDate DATETIME NULL
CONSTRAINT OrderIDMonth PRIMARY
KEY(OrderID, OrderMonth)
)
```

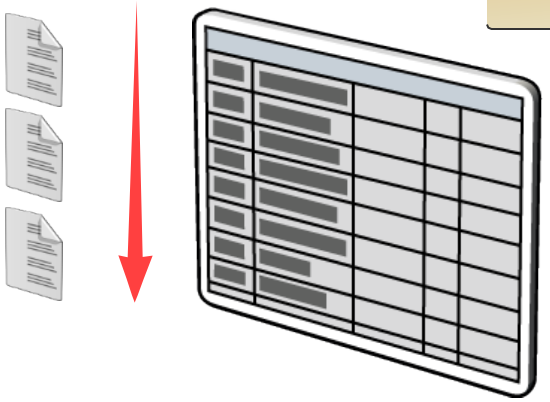
```
CREATE VIEW Year1998Sales
AS
SELECT * FROM Jan1998Sales
UNION ALL
SELECT * FROM Feb1998Sales
UNION ALL
SELECT * FROM Mar1998Sales
UNION ALL
SELECT * FROM Apr1998Sales
UNION ALL
SELECT * FROM May1998Sales
UNION ALL
SELECT * FROM Jun1998Sales
UNION ALL
SELECT * FROM Jul1998Sales
UNION ALL
…
```

Creating and Tuning Indexes

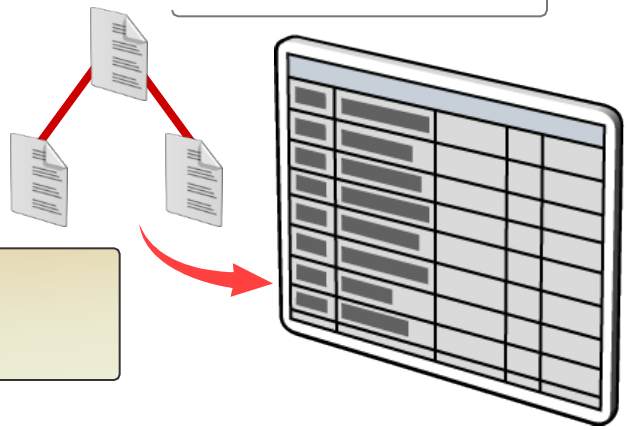
How SQL Server Accesses Data

Table Scan

SQL Server reads all table pages



Index

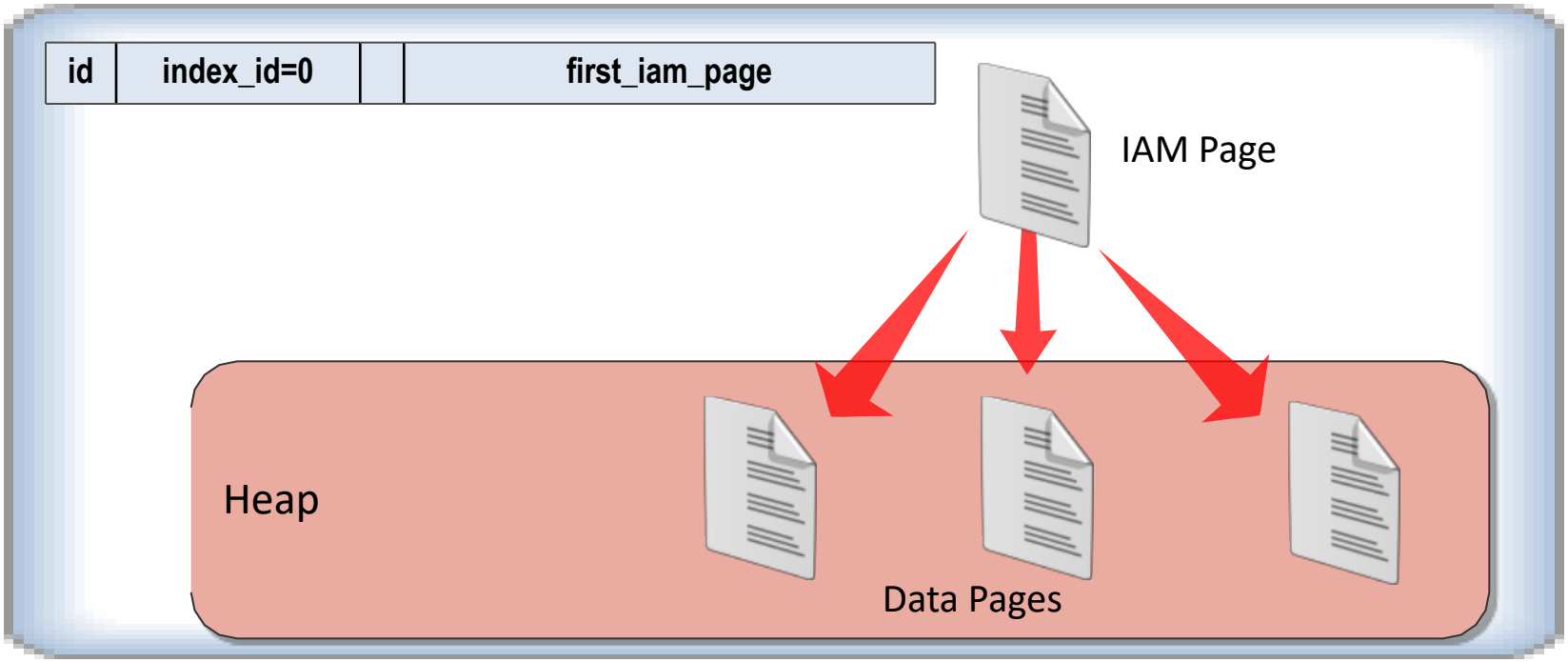


SQL Server uses index pages to find rows

SQL Server 2008 introduces enhanced full-text indexing

What Is a Heap?

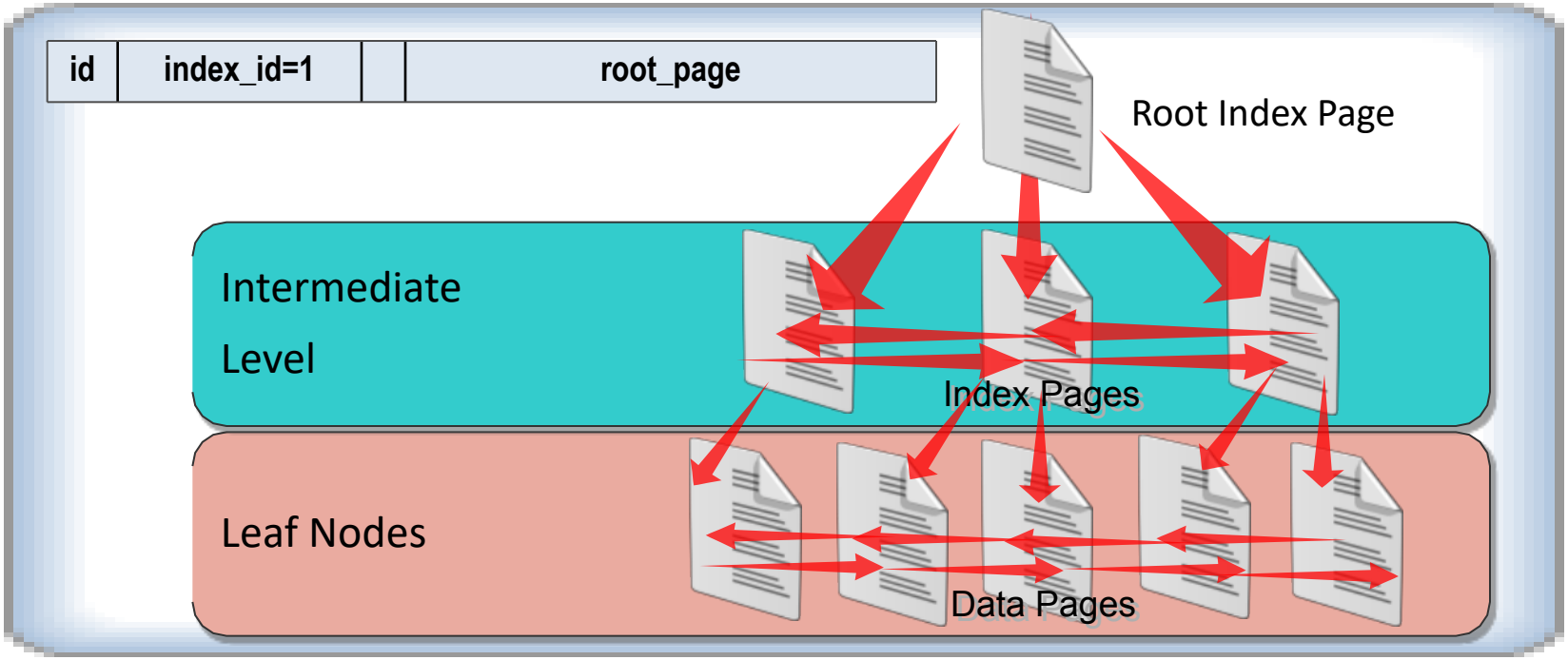
- A table without a clustered index
- Pages stored in no particular order



index Video

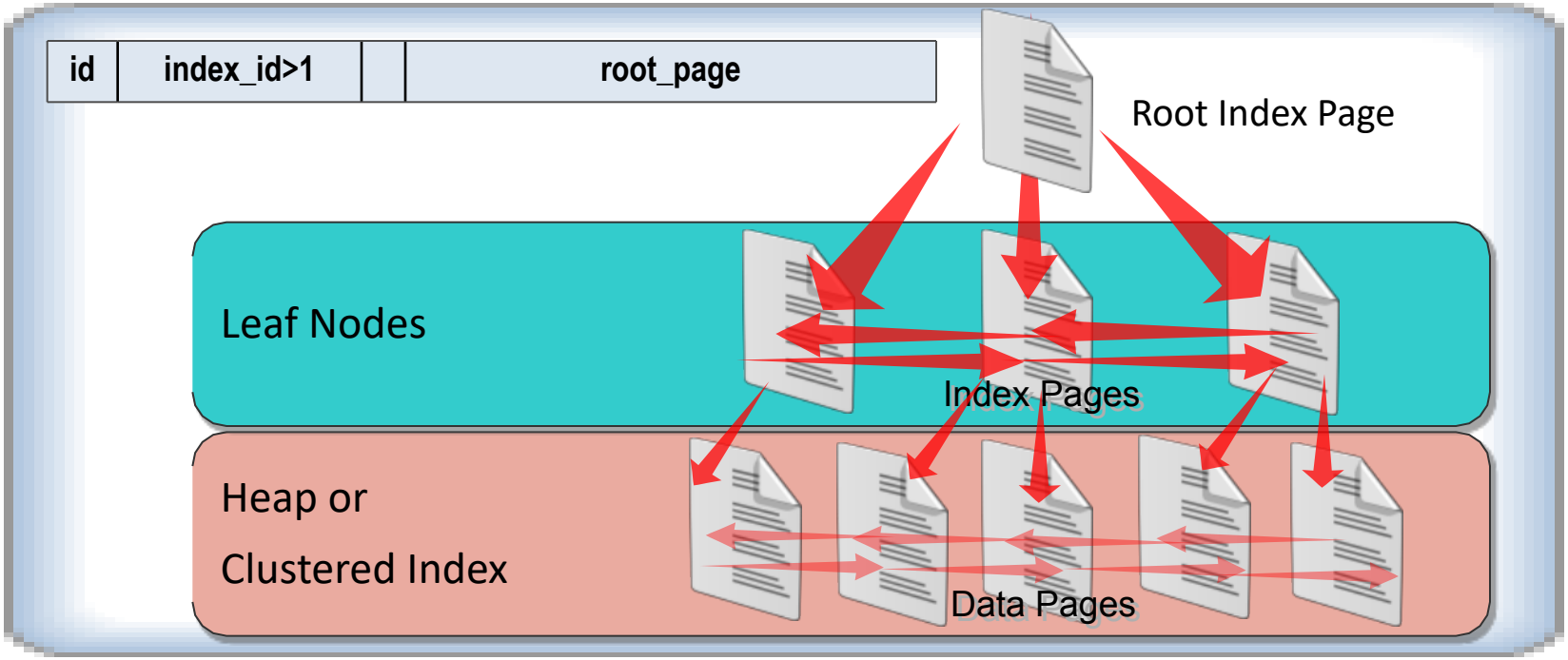
What Is a Clustered Index?

- One clustered index per table
- B-tree stores data pages in order of index key



What Is a Nonclustered Index?

- B-tree references underlying heap or clustered index
- Up to 249 nonclustered indexes per table



Creating Indexes

- Overview of Creating Indexes
- What Are Unique Indexes?
- Considerations for Creating Indexes with Multiple Columns
- When to Create Indexes on Computed Columns
- What Are Partitioned Indexes?
- Options for Incorporating Free Space in Indexes
- Methods for Obtaining Index Information

Overview of Creating Indexes

Use SQL Server Management Studio

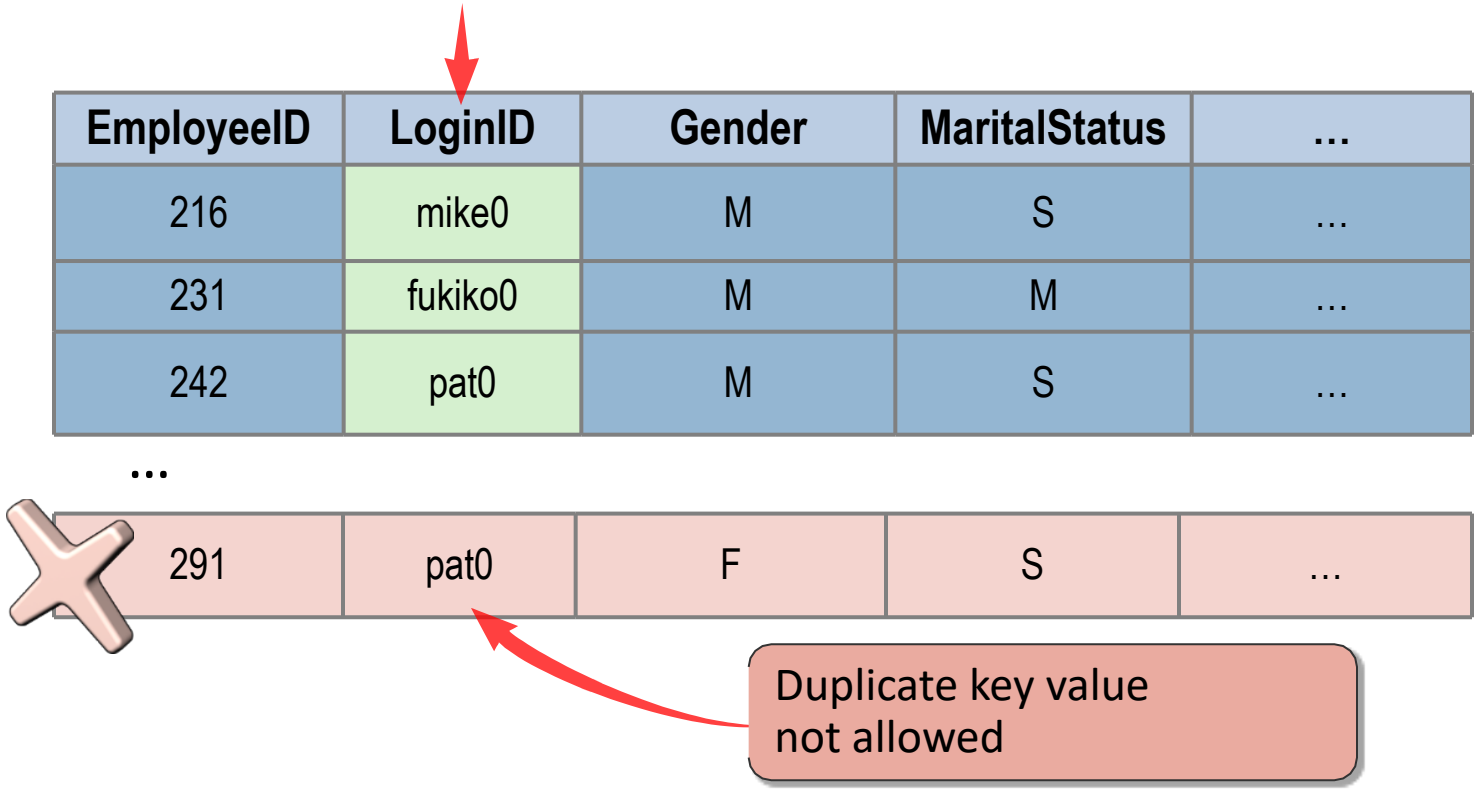
-OR-

CREATE INDEX Transact-SQL statement

```
CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED ]
    INDEX index_name ON { table | view } ( column [ ASC
| DESC]
    [ , . . . n ] )
    INCLUDE ( column [ , . . . n ] )
    [ WITH option [ , . . . n ] ]
    [ ON { partition_scheme (column) | filegroup |
"default" } ]
```

What Are Unique Indexes?

```
CREATE UNIQUE NONCLUSTERED INDEX [AK_Employee_LoginID]
   ON [HumanResources].[Employee] ( [LoginID] ASC )
```



Using Common Table Expressions

- What Are Common Table Expressions?
- Writing Common Table Expressions
- Writing Recursive Queries by Using Common Table Expressions

What Are Common Table Expressions?

Common Table Expression	A named temporary result set based on a SELECT query

- Result set can be used in SELECT, INSERT, UPDATE, or DELETE
- Advantages of common table expressions:
  - Queries with derived tables become more readable
  - Provide traversal of recursive hierarchies

```
WITH TopSales (SalesPersonID, NumSales) AS
( SELECT SalesPersonID, Count(*)
  FROM Sales.SalesOrderHeader GROUP BY SalesPersonId )
SELECT * FROM TopSales
WHERE SalesPersonID IS NOT NULL
ORDER BY NumSales DESC
```

Writing Common Table Expressions

1. Choose a CTE name and column list
2. Create the CTE SELECT query
3. Use the CTE in a query

```
with cte (did,cnt)as
(
select dept_id,COUNT(dept_id)
from Student
group by Dept_Id
)
select *,(select COUNT(*) from cte) from  cte
```

What is the MERGE Statement?

- Transact-SQL command
- Joins a data source with a target table or view
- Performs multiple actions based on the results of the join

How to Use the MERGE Statement

Defines the target table

```
MERGE Production.ProductInventory AS pi
```

Defines the data source

```
USING (SELECT ProductID, SUM(OrderQty) FROM
    Sales.SalesOrderDetail sod
    JOIN Sales.SalesOrderHeader soh
    ON sod.SalesOrderID = soh.SalesOrderID
    AND soh.OrderDate = GETDATE()
    GROUP BY ProductID) AS src (ProductID, OrderQty)
```

Defines the value used to match rows

```
ON (pi.ProductID = src.ProductID)
```

Defines the action to take based on the results from the ON clause

```
WHEN MATCHED AND pi.Quantity - src.OrderQty <> 0
    THEN UPDATE SET pi.Quantity = pi.Quantity - src.OrderQty
WHEN MATCHED AND pi.Quantity - src.OrderQty = 0
    THEN DELETE; --Update qty or delete product when qty = 0
```

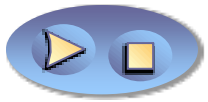
Temporary Tables

- Local Temporary Tables:
  - Have a single number sign (#) as the first character of their names
  - Visible only to the current connection for the user
  - Deleted when the user disconnects from SQL Server

```
CREATE TABLE #StoreInfo
(
  EmployeeID int,
  ManagerID int,
  Num int
)
```

- Global Temporary Tables:
  - Have a double number sign (##) as the first character of their names
  - Visible to any user once created
  - Deleted when all users referencing them disconnect

```
CREATE TABLE ##StoreInfo
(
  EmployeeID int,
  ManagerID int,
  Num int
)
```



Uses for Temporary Tables

- Uses for temporary tables:
  - By a user:
    - Hold the results of queries that are too complex for a single query
    - Hold the intermediate work of a stored procedure
  - By the system:
    - Hold the intermediate results of a query that requires ordering or elimination of nondistinct values
    - Hold the intermediate results of a query that has an aggregate or **compute** clause

Shareable Temporary Tables

- Created explicitly in *tempdb*
  - Example 1:

```
use tempdb
go
create table temp_table (
        pub_id          char(4)         NOT NULL,
        pub_name        varchar (30)    NULL)
```

  - Example 2:

```
create table tempdb..temp_table (
        pub_id          char(4)         NOT NULL,
        pub_name        varchar(30)     NULL)
```

- Accessible to any user
- Exist in *tempdb* until one of the following occurs:
  - The table is explicitly dropped via **drop table**
  - The server is rebooted

Session-Specific Temporary Tables

- Created by prefixing a table name with a #
  - Example:

```
create table #temp_table (
        pub_id          char(4)         NOT NULL,
        pub_name        varchar(30)     NULL)
```

- Accessible only by the session (user) that created the table
- Exist in *tempdb* until one of the following occurs:
  - The table is explicitly dropped via **drop table**
  - The session ends (the user is disconnected from the server)

Temporary Tables: Summary

	How is it created?	Who can access it?	When will the server drop it?
Shareable	By creating the table explicitly in tempdb	Any user	When the server is rebooted
Session-specific	By prefixing a table name with a #	Only the user who created it	When the session ends

Subqueries versus Temporary Tables

- As subqueries get more complex their performance may decrease
- Maintainability can be easier with subqueries in some situations, and easier with temporary tables in others
- Temporary tables can be easier for some to debug while others prefer to work with a single subquery

System Table

- A system table is a table created and maintained by the server that stores information about the server or one of its databases
- Examples of system tables:
  - ***sysobjects***
    - One row for each table, view, rule, default, procedure, trigger, log, and (in *tempdb* only) temporary object in the given database
  - ***sysmessages***
    - One row for each message used by the server

Querying System Tables

- Use the **select** statement as if the system table were a user table
- Example that lists information on all views in a database:

```
select * from sysobjects
where type = "V"
```

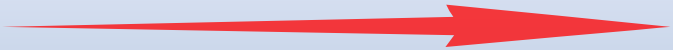
- Example that lists all users in a database:

```
select name from sysusers
```

How the PIVOT and UNPIVOT Operators Work

- PIVOT – converts values to columns

Cust	Prod	Qty
Mike	Bike	3
Mike	Chain	2
Mike	Bike	5
Lisa	Bike	3
Lisa	Chain	3
Lisa	Chain	4



Cust	Bike	Chain
Mike	8	2
Lisa	3	7

- UNPIVOT – converts columns to values

Cust	Bike	Chain
Mike	8	2
Lisa	3	7

Cust	Prod	Qty
Mike	Bike	8
Mike	Chain	2
Lisa	Bike	3
Lisa	Chain	7

Using the PIVOT Operator

- PIVOT – converts values to columns

Cust	Prod	Qty
Mike	Bike	3
Mike	Chain	2
Mike	Bike	5
Lisa	Bike	3
Lisa	Chain	3
Lisa	Chain	4

```
SELECT * FROM Sales.Order
PIVOT (SUM(Qty) FOR Prod IN
([Bike],[Chain])) PVT
```

Cust	Bike	Chain
Mike	8	2
Lisa	3	7

Using the UNPIVOT Operator

- UNPIVOT – converts columns to values

Cust	Bike	Chain
Mike	8	2
Lisa	3	7

```
SELECT Cust, Prod, Qty
FROM Sales.PivotedOrder
UNPIVOT (Qty FOR Prod IN
([Bike],[Chain])) UnPVT
```

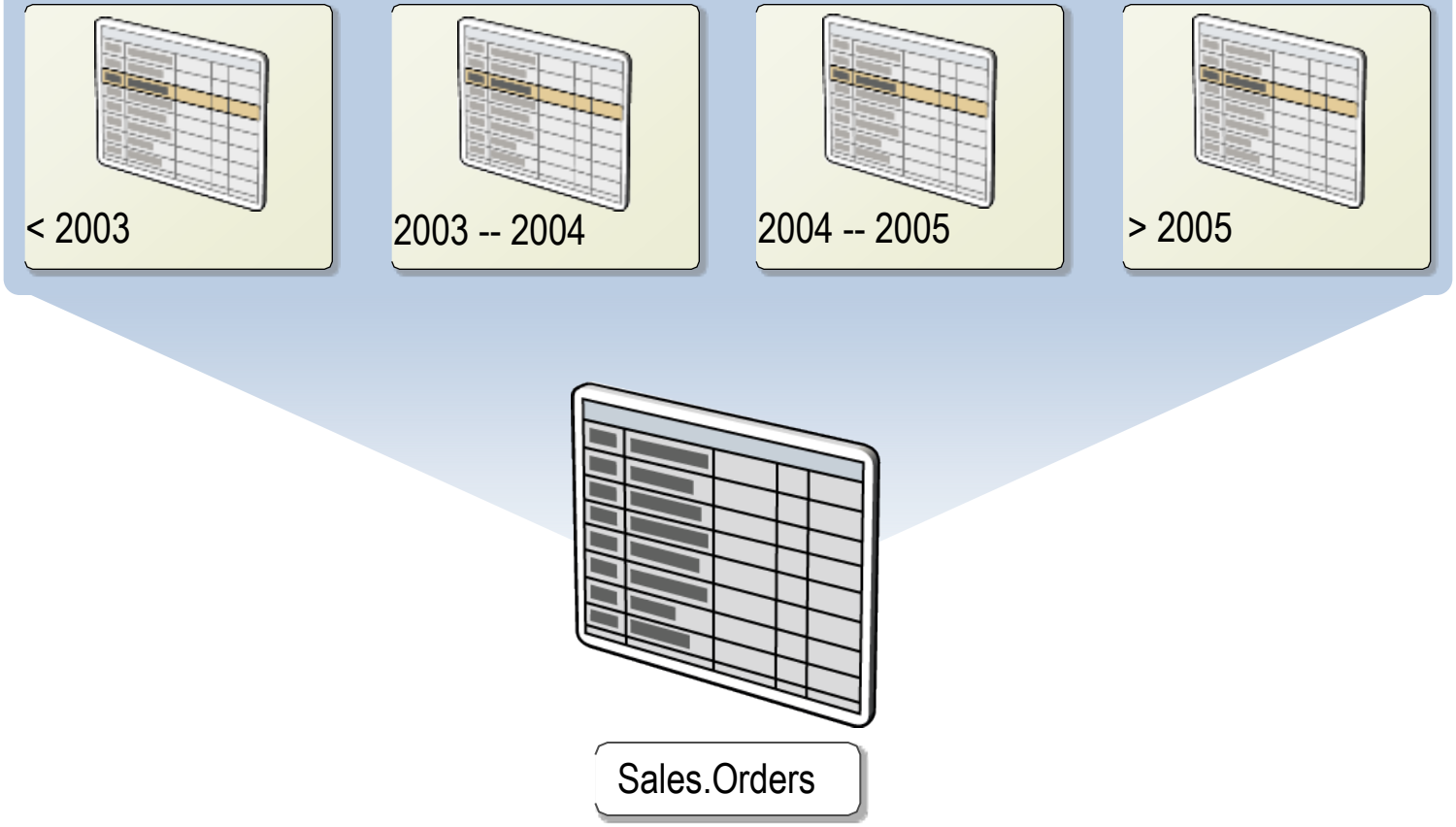
Cust	Prod	Qty
Mike	Bike	8
Mike	Chain	2
Lisa	Bike	3
Lisa	Chain	7

Creating Partitioned Tables

- What Are Partitioned Tables?
- What Are Partition Functions?
- What Is a Partition Scheme?
- What Operations Can Be Performed on Partitioned Data?

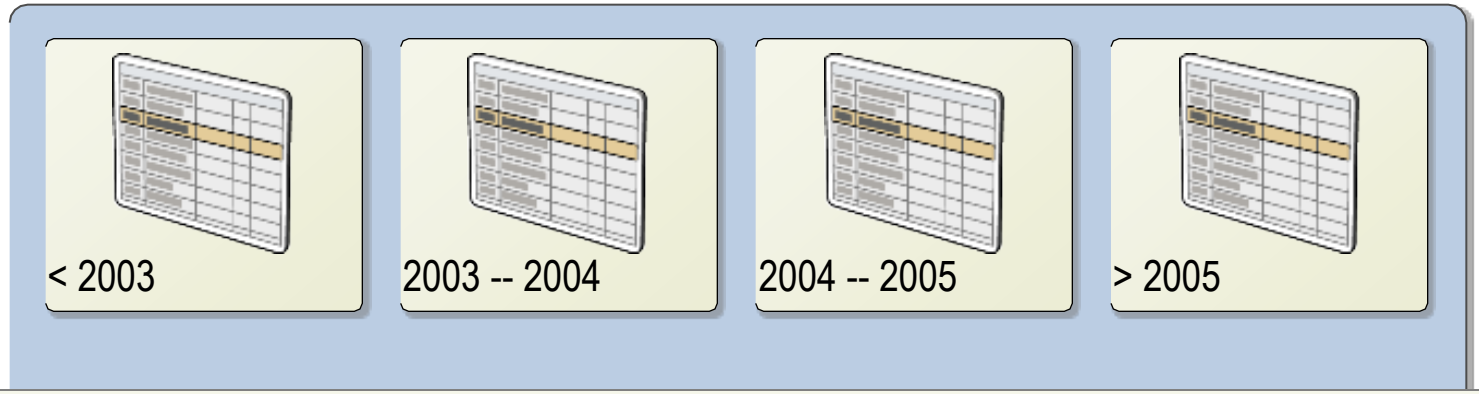
What Are Partitioned Tables?

Data is partitioned horizontally by range



What Are Partition Functions?

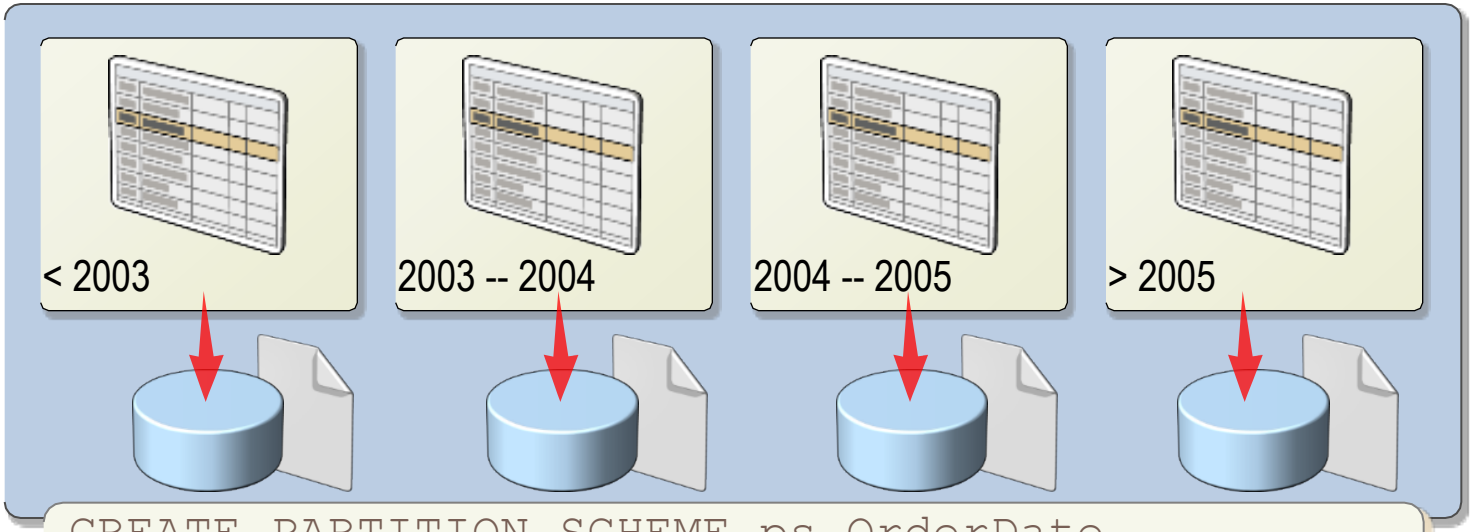
- Partition functions define partition boundaries
- Boundary values can be assigned to LEFT or RIGHT




```
CREATE PARTITION FUNCTION pf_OrderDate (datetime)
AS RANGE RIGHT
FOR VALUES ('01/01/2003', '01/01/2004', 01/01/2005')
```

What Is a Partition Scheme?

- A partition scheme assigns partitions to filegroups
- The “next” filegroup can also be defined




```
CREATE PARTITION SCHEME ps_OrderDate
AS PARTITION pf_OrderDate
TO (fg1, fg2, fg3, fg4, fg5)
```